Large Language Model-driven Large Neighborhood Search for Large-Scale MILP Problems

Huigen Ye, Hua Xu, An Yan, Yaoyang Cheng, ICML 2025 spotlight

Presenter: Jongmin Mun

USC ISE-617

Large Neighborhood Search of Mixed Integer LP

- Scalability challenge: Branch-and-bound struggles with large MILP instances.
- **Large Neighborhood Search (LNS)** starts from a feasible solution and iteratively improves solutions via Destroy and Repair operators.
- Bottleneck: The Destroy phase. Success depends heavily on choosing which variables to fix vs. free.

Existing strategies and limitations

- Handcrafted Heuristics: Labor-intensive and task-specific. Fails to easily generalize to new problems (*cold-start issue*).
- Reinforcement Learning : Intractable search spaces. Trial-and-error exploration of RL agent is highly inefficient, leading to *slow convergence*.
- Imitation Learning : Requires massive amounts of high-quality labels generated by a *computationally expensive* oracle.
- LLM-based evolution (e.g., FunSearch, EOH): Reliance on fixed evolutionary prompts often leads to **premature convergence**.

Proposed Approach: The Dual-Layer Agent

The core contribution is a very smart prompt engineering

- **The Baseline:** Prior LLM methods only guide heuristic evolution with a fixed prompt (only for combinatorial problems, not MILP)
- **The Bottleneck:** Fixed evolutionary prompts restrict exploration and cause premature convergence.
- **The Solution:** A co-evolutionary, two-LLM architecture:
  - **Inner Layer (Heuristic Evolution):** Given a fixed prompt, generates and refines heuristics for rapid convergence
  - **Outer Layer (Prompt Evolution):** Improves the prompt to force new **exploration**
  - **Explicit memory**: in the inner layer, the prompt explicitly states to learn from history (both from good and bad examples)

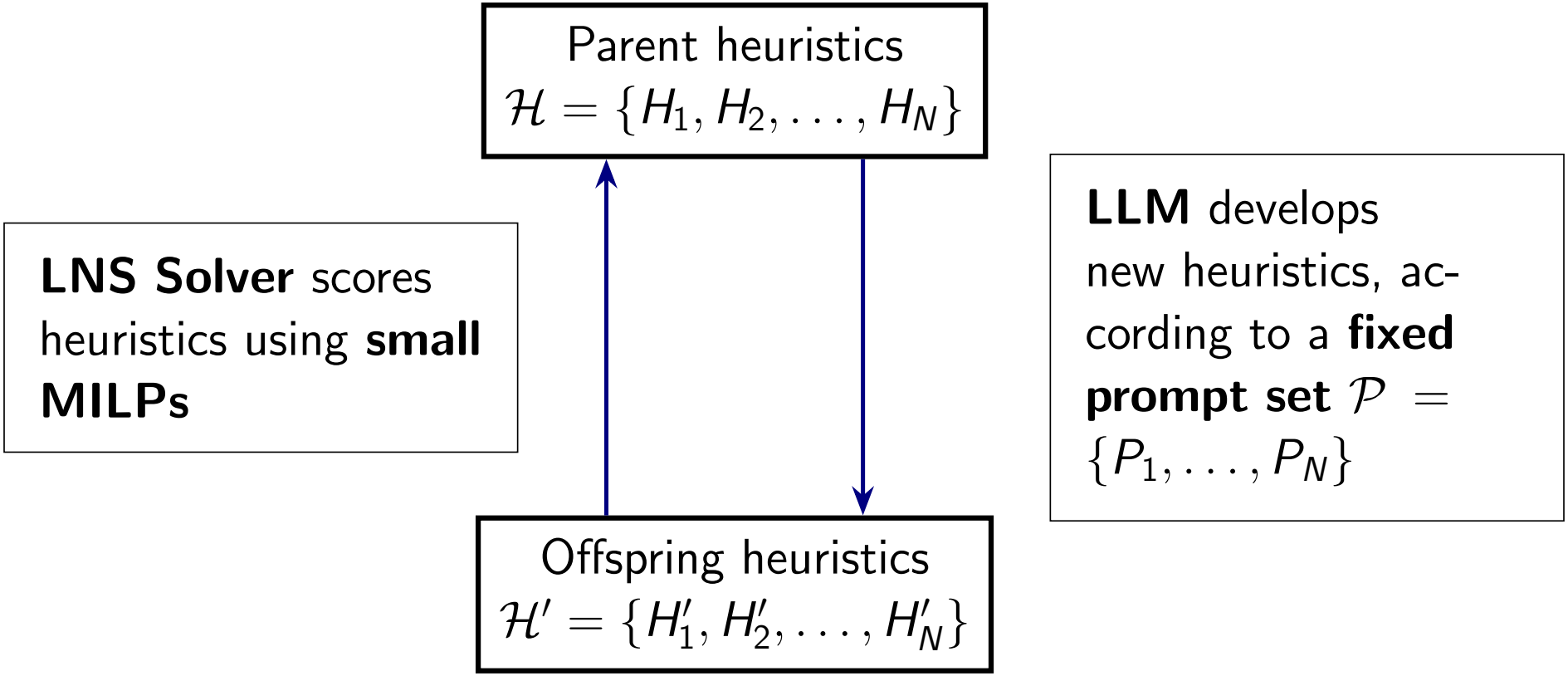
Dual-Layer Agent: Inner Layer (Heuristic Evolution)

The **Inner Layer** focuses on evolving heuristic strategies (thoughts and code) to ensure rapid convergence[cite: 2054, 2162].

- Prior LLM approaches established that language models can guide heuristic evolution.
- The Limitation: Fixed evolutionary prompts limit exploration.
- This Paper's Approach: Use two LLMs. One evolves the heuristics. The other evolves that LLM.

Inner Exploitation Loop: Evolution of Heuristics

Exploitation: find the best possible heuristics with evolution prompt fixed



Repeat for a fixed number of time; the whole procedure can be written as

$$\mathcal{H} = \texttt{inner}(\texttt{prompt})$$

- Generation 0: capacity ratio with penalties

$$\text{scores}[\text{bins} > \text{item}] = \frac{\text{bins} - \text{item}}{\text{bins}} \times \frac{1}{\text{index of bin} + 0.5}$$

- Generation 5: Random exploration factor added

$$\text{scores}[\text{bins} > \text{item}] = \frac{\text{bins} - \text{item}}{\text{bins} + 10^{-5}} - \frac{\frac{\text{bins}^2}{10^2}}{\max\left(\frac{\text{bins}^2}{10^2}\right)} \left(1 - \frac{\text{bins} - \text{item}}{\max(\text{bins})}\right) + \text{randomness} \times 0.1$$

- Generation 8: Advanced hybrid optimization methods, such as genetic algorithms combined with tabu search, are introduced

$$\text{scores}[\text{bins} > \text{item}] = \frac{\text{bins} - \text{item}}{\text{bins} + 10^{-5}} - \text{history penalty} - \frac{\frac{\text{bins}^2}{15^2}}{\max\left(\frac{\text{bins}^2}{15^2}\right)} + \text{mutation variance} - \left(1 - \frac{\text{bins} - \text{item}}{\max(\text{bins}) + 10^{-5}}\right)$$

Outer Exploration Loop: Evolution of Prompts

Exploration: when inner loop stagnates, change the prompt

Algorithm 1 Outer loop for exploration

Require: Initial heuristic H_0, Initial prompt P, Total iterations T

```
for t = 1 to T do
    H_t ← inner(P)
    if no improvement of H_t for a certain number of steps then
        P ← LLMevolve(P)
    end if
end for
```

Final step: use the best strategy out of $\mathcal{H}_T$ and run LNS on massive MILP.

- **Generation 1: Initialization by handcrafted prompts** "Create a completely new algorithm", "Modify the given algorithm"
- **Generation 6: Addressing Stagnation.** "Reconfigure heuristic principles to **reduce computational complexity** while maintaining effectiveness."
- **Generation 10: Stochastic Adjustment.** "Enhance diversity by **introducing stochastic elements** to refine exploration strategies."

Inside the LLM: Differential Memory for Directional Evolution

- **Historical Context:** Inside the inner loop, LLM reads a set of m strategy-thought-fitness tuples, capturing both successful and unsuccessful heuristics:

$$S^{(t)} = \left\{ \langle H_i^{(t)}, \text{thought}_i, f(H_i^{(t)}) \rangle \right\}_{i=1}^{m}$$

- **Directional Evolution:** The `LLM` generates the next iteration by conditioning on this historical memory:

$$H_i^{(t+1)} = \texttt{LLM}(\texttt{prompt} \parallel S^{(t)})$$

- **The** `prompt` combines two key components:
  - The strategic prompt provided by the Outer Loop.
  - A differential memory instruction: *"Learn from history, both good and bad. Think of what makes the difference."*

Experimental Setup: Task 1

Task 1: Combinatorial Optimization

- Problems: Online Bin Packing, Traveling Salesman Problem (TSP)
- Baselines:
  - Hand-crafted heuristics
  - Neural Networks (POMO, LEHD)
  - LLM-based approaches (FunSearch, EOH)

Results: Combinatorial Optimization (bin packing)

500 bins instances:

Method / items	**1k**	**5k**	**10k**	**Avg**
First Fit	4.97%	4.27%	4.28%	4.61%
Best Fit	4.50%	3.91%	3.95%	4.23%
FunSearch	6.75%	1.47%	0.74%	2.31%
EOH	4.32%	1.06%	0.97%	2.09%
LLM-LNS	**3.67%**	**0.82%**	**0.42%**	**1.63%**

- LLM-LNS outperforms handcrafted heuristics in all instances.
- LLM-LNS outperforms existing LLM approaches in large instances.
- Not on the table, but also beats ML based methods oh large instances

Experimental Setup: Task 2

Task 2: Large-Scale MILP Benchmark

- Datasets: Set Covering (SC), Minimum Vertex Cover (MVC), Maximum Independent Set (MIS), Mixed Integer Knapsack Set (MIKS)
- Baselines: ACP, CL-LNS, Gurobi, SCIP, GNN&GBDT, Light-MILPOPT.
- No LLM-based baselines.

Results: Large-Scale MILP Generalization

Method	MIS		MIKS	
	MIS1 (Max)	**MIS2** (Max)	**MIKS1** (Max)	**MIKS2** (Max)
Random-LNS	22892.9	223748.6	36011.0	351964.2
ACP	23058.0	226498.2	34190.8	332235.6
CL-LNS	15000.0	-	-	-
Gurobi	21789.0	216591.3	32960.0	329642.4
SCIP	18649.9	9104.3	29974.7	168289.9
GNN&GBDT	22795.7	227006.4	*	*
Light-MILPOPT	22966.5	230432.9	36125.5	362265.1
LLM-LNS	**23169.3**	**231636.9**	**36479.8**	**363749.5**

Table: Performance comparison on MIS and MIKS benchmarks. Best results are highlighted in bold.

MILP efficiency

Measured by primal integral, LLM-LNS is at least 20% more efficient than the closest competitors

Conclusion

- **The Gap:** While LLM-based heuristics overcome the data and convergence limits of Imitation Learning and RL, fixed prompts often trap them in **local optima**.
- **The Contribution:** A **Dual-Layer Agent** architecture.
  - *Outer Loop:* Dynamically evolves prompts to force exploration.
  - *Inner Loop:* Evolves heuristics by explicitly learning from both successful and failed historical attempts.
- Empirically outperforms existing state-of-the-art methods on large-scale problems in both **objective quality** and **efficiency**, while remaining **easy to implement**.

Limitations

- Heuristics are learned from and evaluated on small-sized problems. Cannot explain why it generalizes to big problems.
- Computationally efficient, but ignores high-quality labeled data.